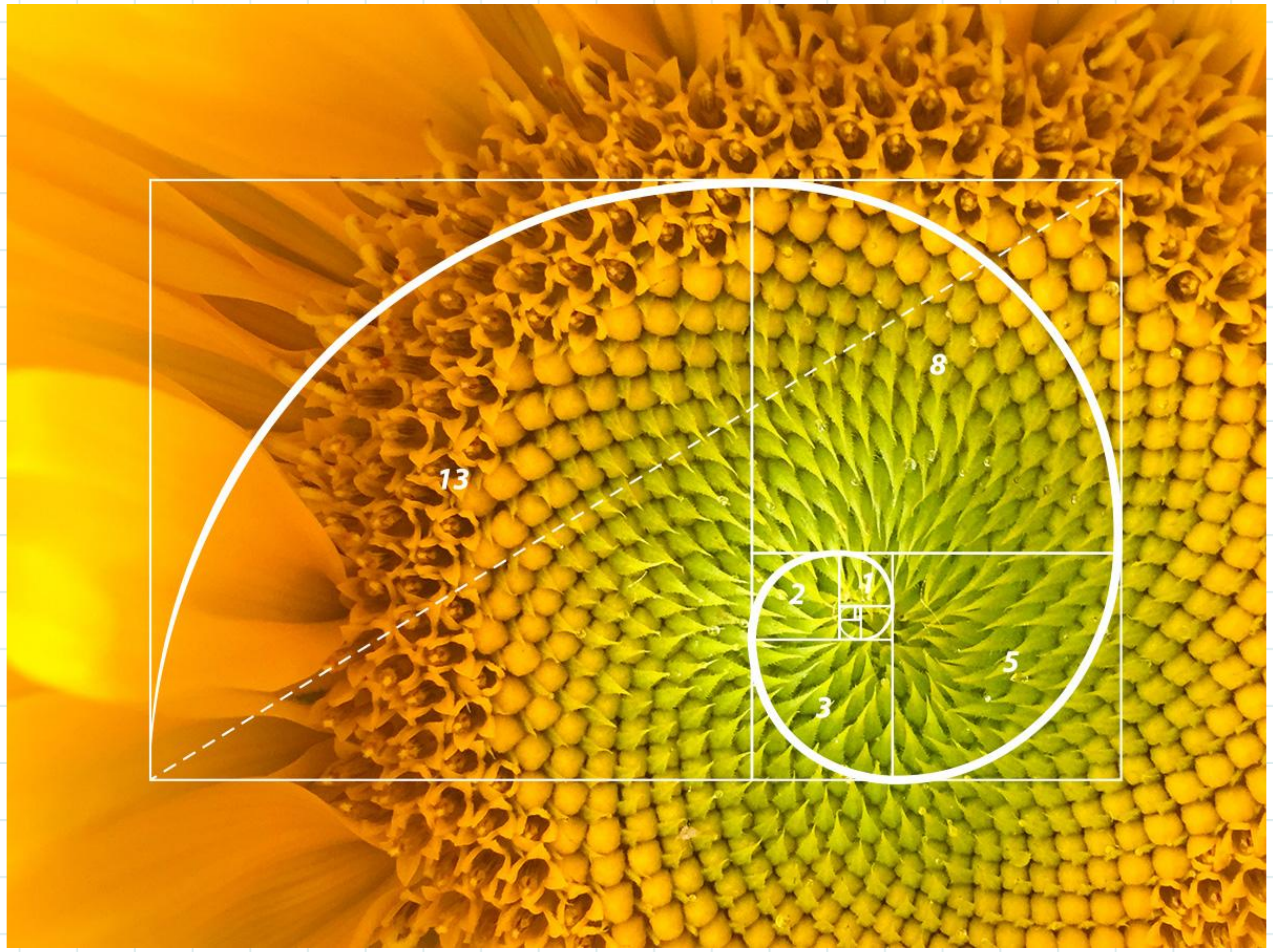
MACHINE LEARNING

K-Means Clustering



Dr. Sirojul Munir, S.Si., M.Kom.
rojulman@nurulfikri.ac.id

ARTIFICIAL INTELLIGENCE – INFORMATICS STTNF

Daur ulang Project Data Science

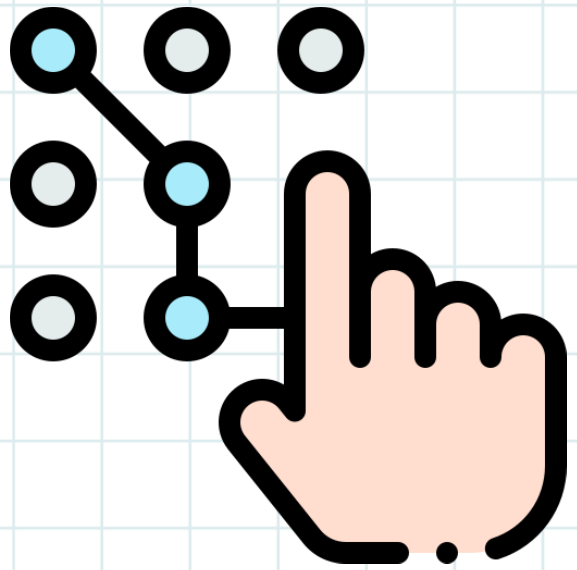
Algoritma *Clustering*

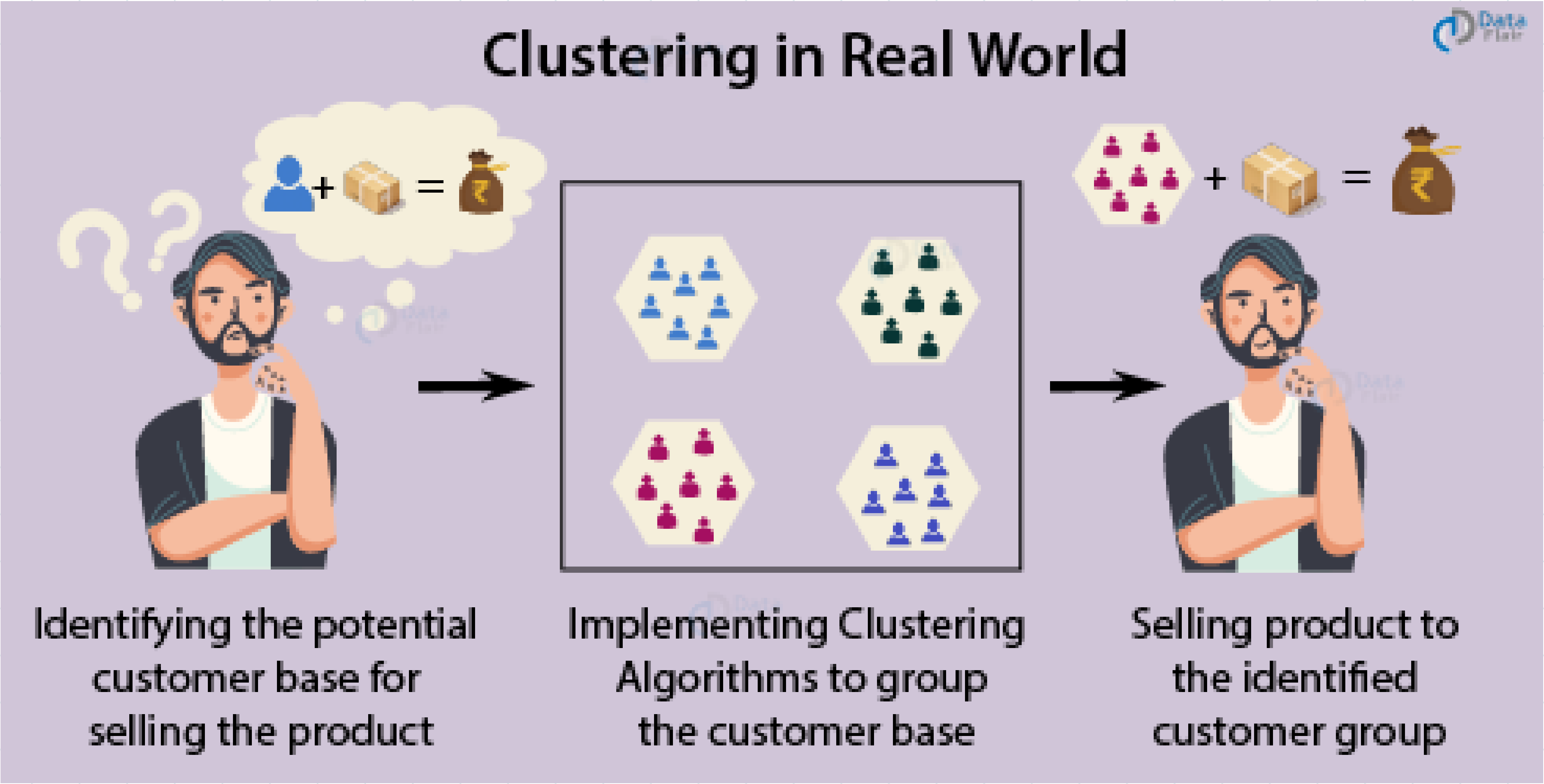
Teknik data mining yang digunakan untuk **mengorganisasi** dan **mengelompokkan** data menjadi beberapa kelompok atau klaster berdasarkan **kesamaan fitur** atau **atribut** tertentu

https://www.analyticsvidhya.com

Tujuan *Clustering*

1. **mengindentifikasi data tidak berlabel** dalam dataset untuk dimasukan dalam suatu klaster data berdasarkan spesifik kriteria

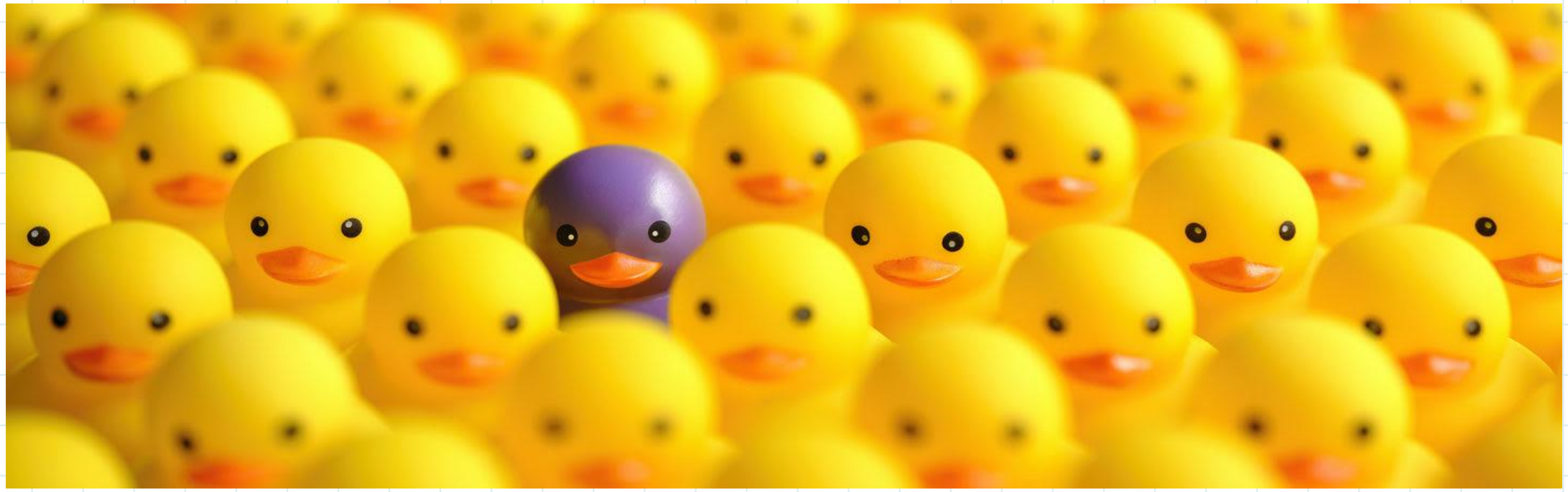




Tujuan *Clustering*

2. **Segmentasi Gambar**
   untuk sistem deteksi dan pelacakan objek

3. **Anomali Detection**
   untuk deteksi berdasarkan prilaku abnormal user pasa sistem keamanan

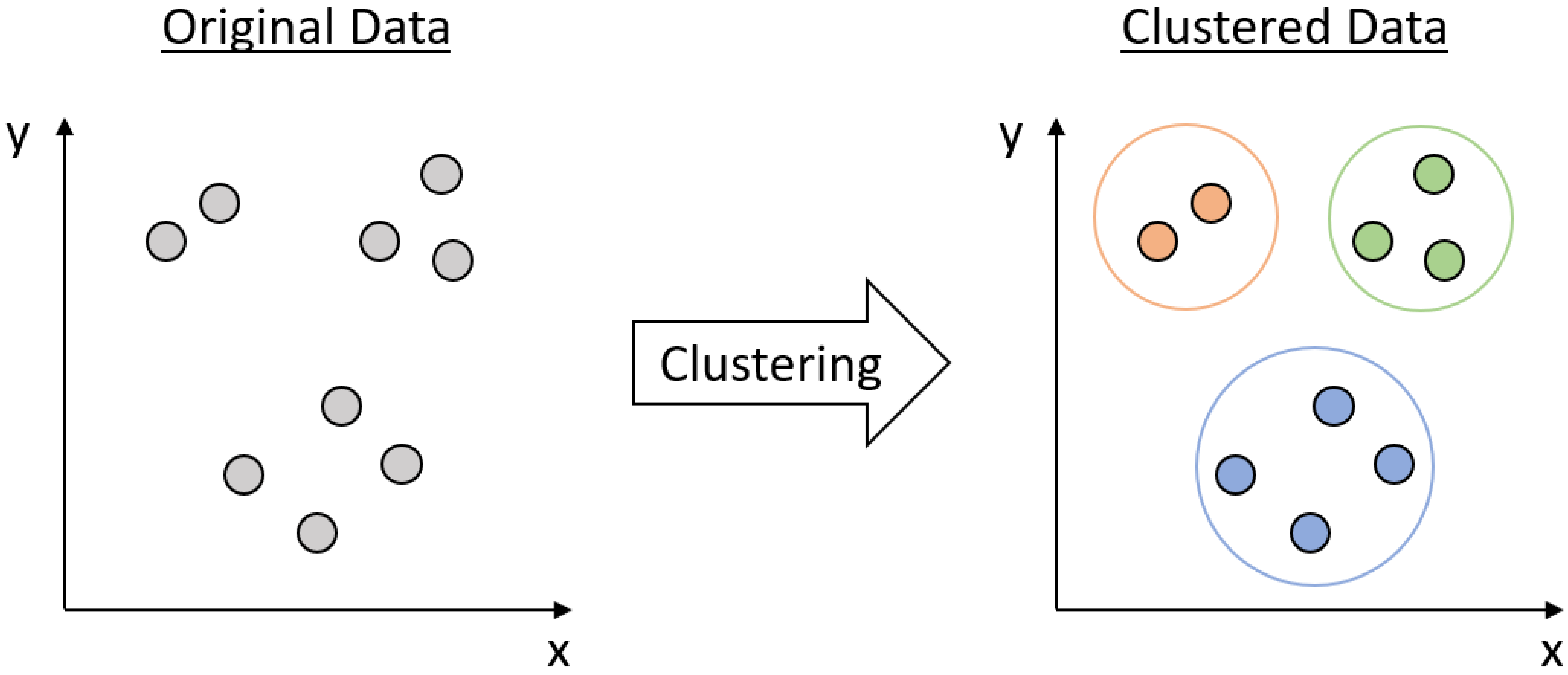


4. **Segmentasi Pelanggan**
   untuk clustering berdasarkan history belanja / pembayaran

https://www.analyticsvidhya.com

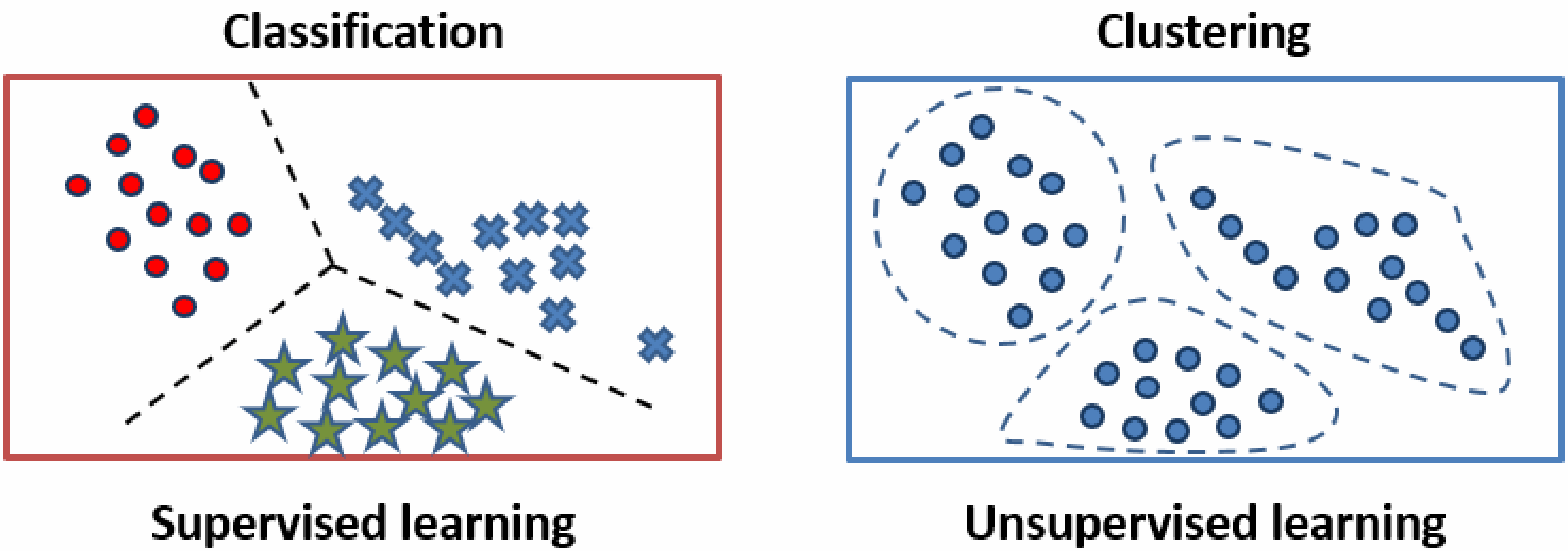
Algoritma ***Clustering***

- *Clustering = Cluster Analysis = Unsupervised Machine Learning*



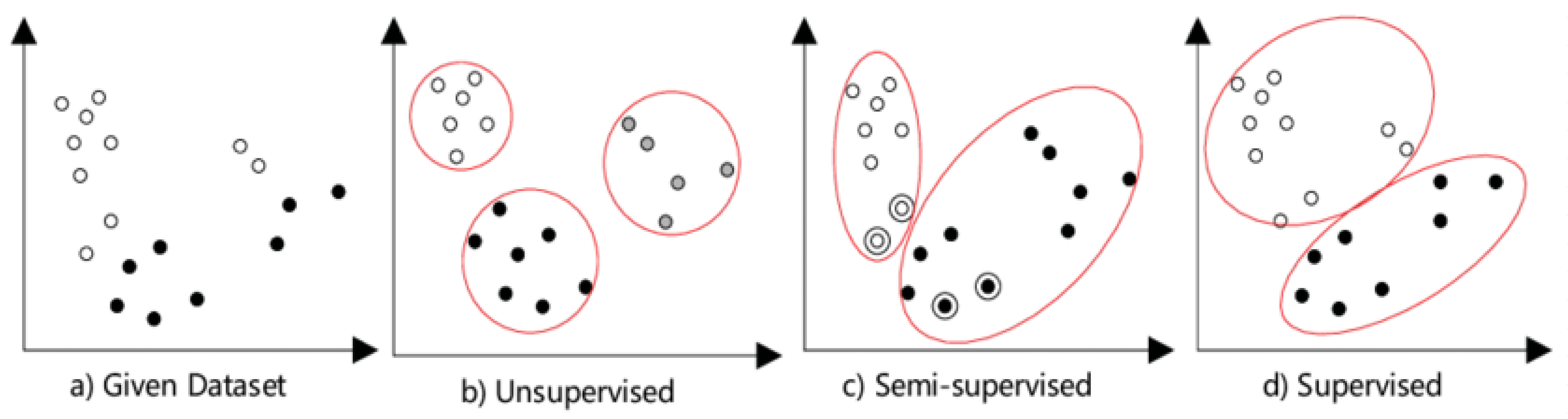
Algoritma *Clustering*

- *Clustering = Cluster Analysis = Unsupervised Machine Learning*



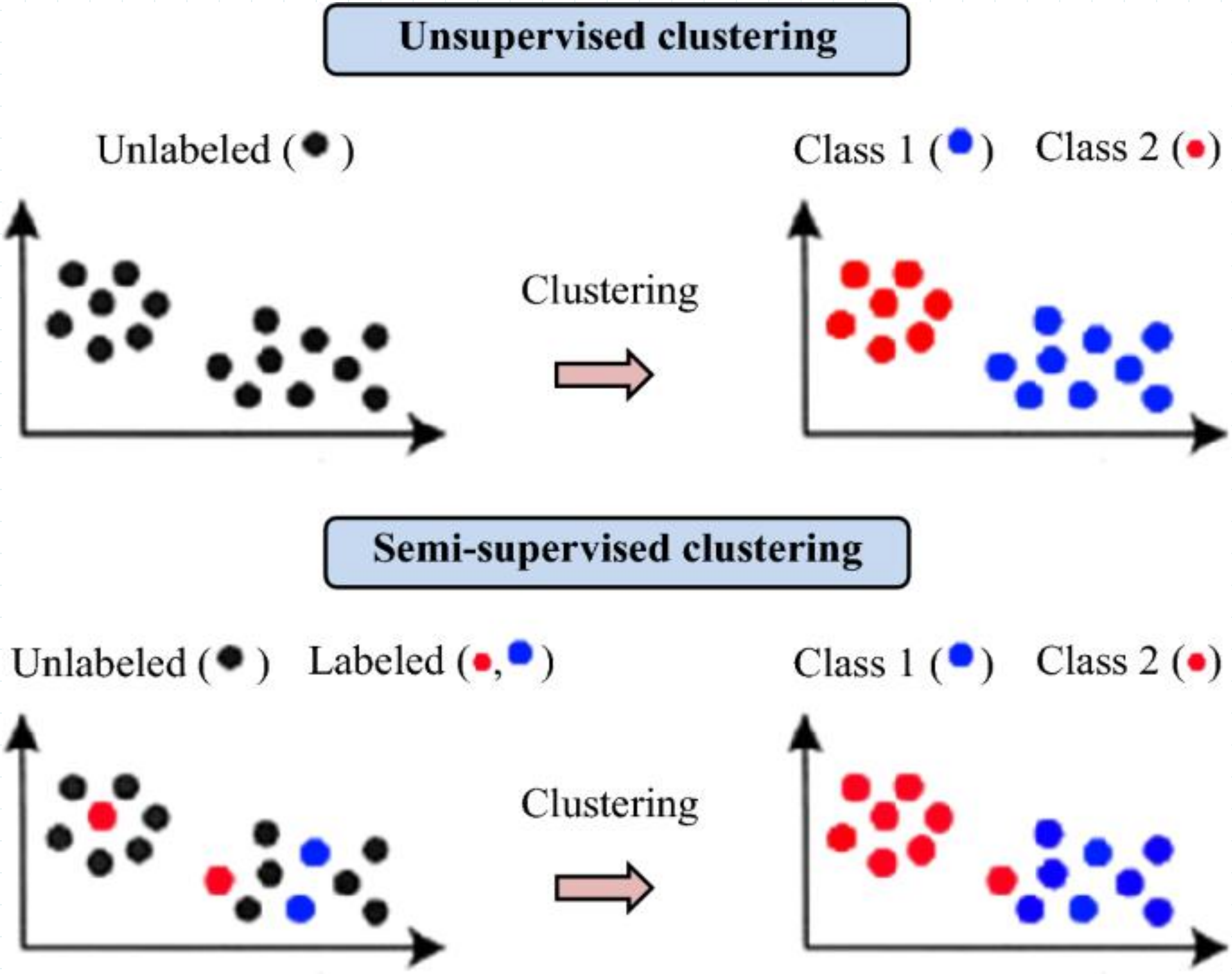
Algoritma *Clustering*

- *Clustering* = *Cluster Analysis* = Semi *supervised Machine Learning*



Clustering

Semi *supervised*
Machine Learning

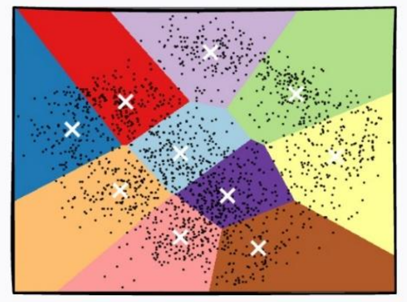
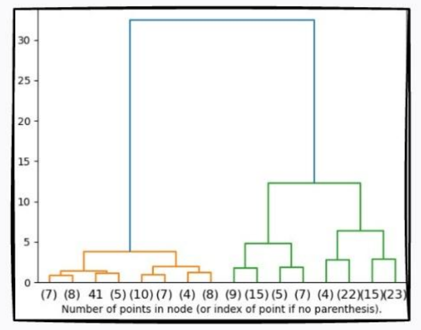
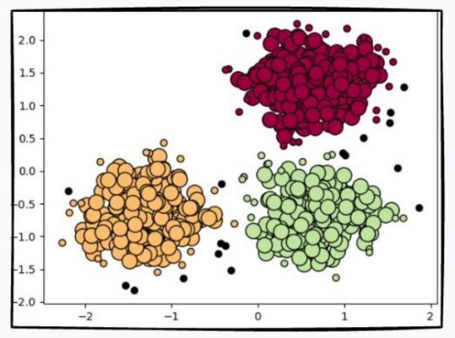
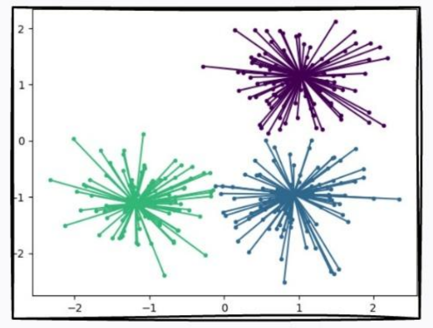
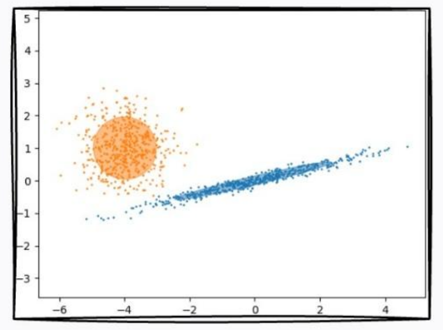
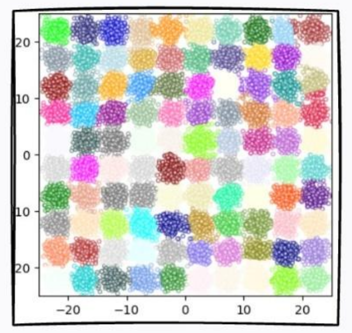


Tipe Algoritma Clustering

1. Centroid-based
2. Connectivity-based
3. Density-based
4. Graph-based
5. Distribution-based
6. Compression-based

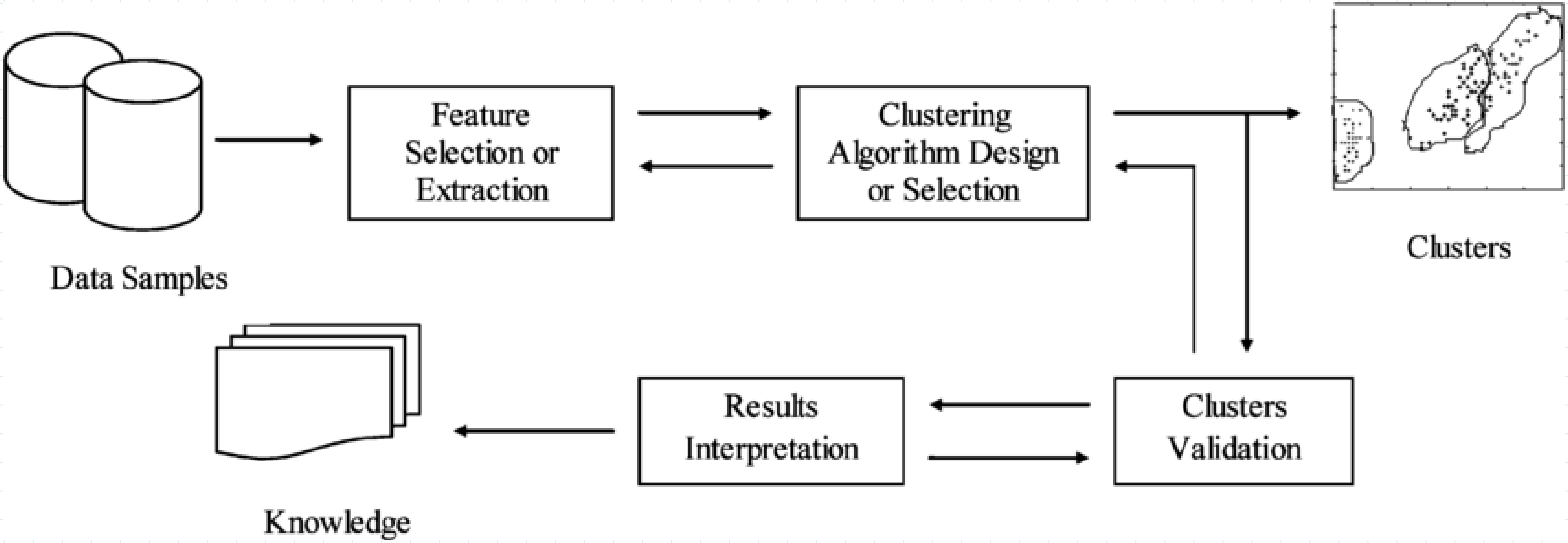
6 Types of Clustering Algorithms in Machine Learning

blog.DailyDoseofDS.com

Clustering Algorithm Type		Clustering Methodology	Algorithm(s)
	Centroid-based	Cluster points based on proximity to centroid	KMeans KMeans++ KMedoids
	Connectivity-based	Cluster points based on proximity between clusters	Hierarchical Clustering (Agglomerative and Divisive)
	Density-based	Cluster points based on their density instead of proximity	DBSCAN OPTICS HDBSCAN
	Graph-based	Cluster points based on graph distance	Affinitiy Propagation Spectral Clustering
	Distribution-based	Cluster points based on their likelihood of belonging to the same distribution.	Gaussian Mixture Models (GMMs)
	Compression-based	Transform data to a lower dimensional space and then perform clustering	BIRCH

STT-NF

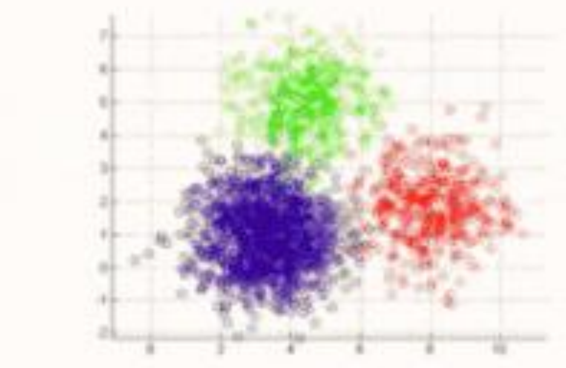
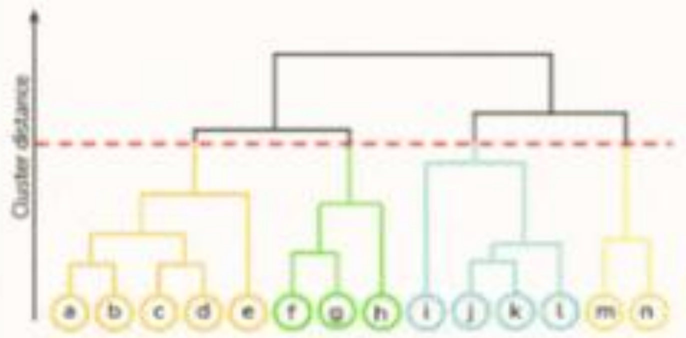
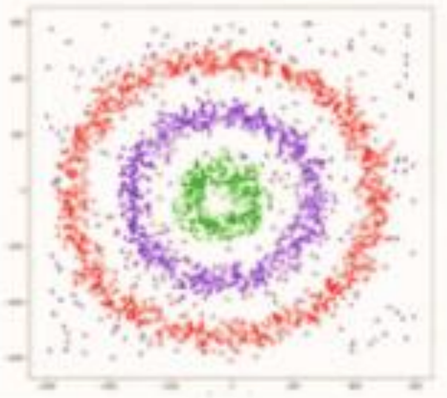
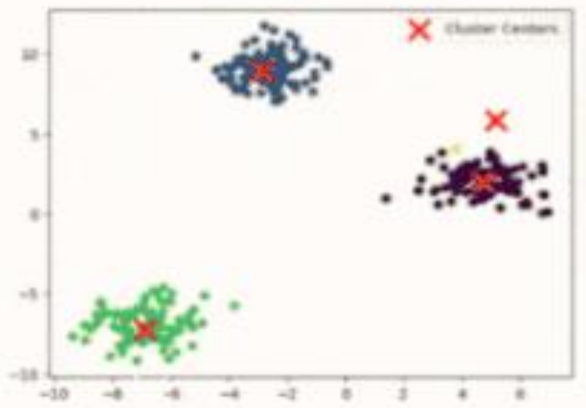
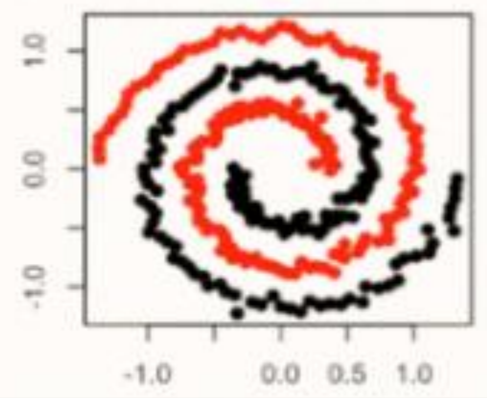
Tahapan *Clustering*



Algoritma Clustering

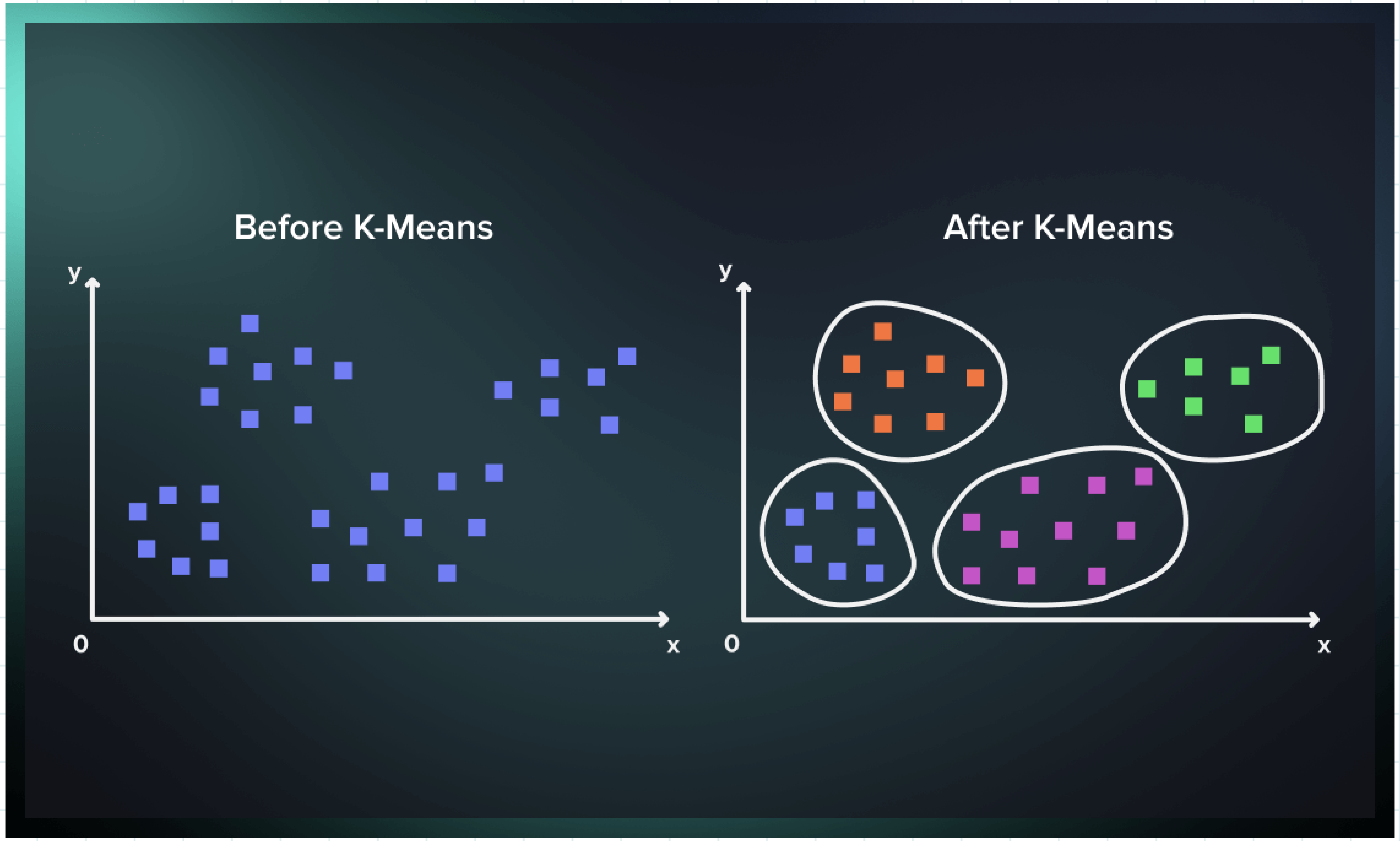
1. **K- Means Clustering**
2. Hierarchical Clustering
3. DBSCAN
4. Mean Shift
5. Spectral Clustering

Top 5 Clustering Techniques

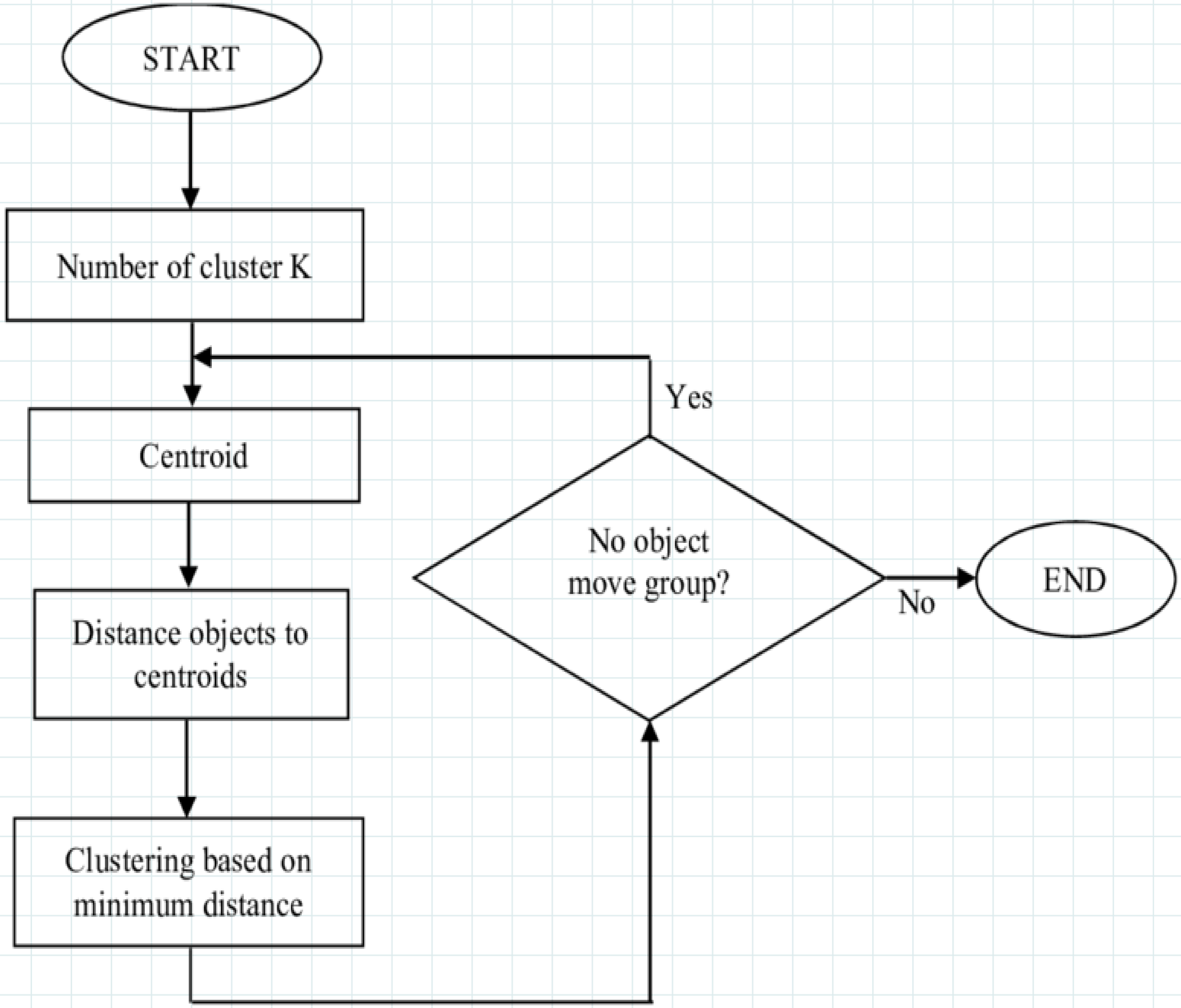
Representation	Algorithm Name	Hyperparameter
	K- Means Clustering	Partitions data into K clusters by minimizing variance within each cluster.
	Heirarchical Clustering	Builds a hierarchy of clusters by iteratively merging or splitting existing groups.
	DBSCAN	Forms clusters based on density; groups densely packed points and marks outliers.
	Mean Shift	Finds clusters by locating and adapting to the centroids of data points.
	Spectral Clustering	Uses eigenvalues of similarity matrix to reduce dimensions before clustering.

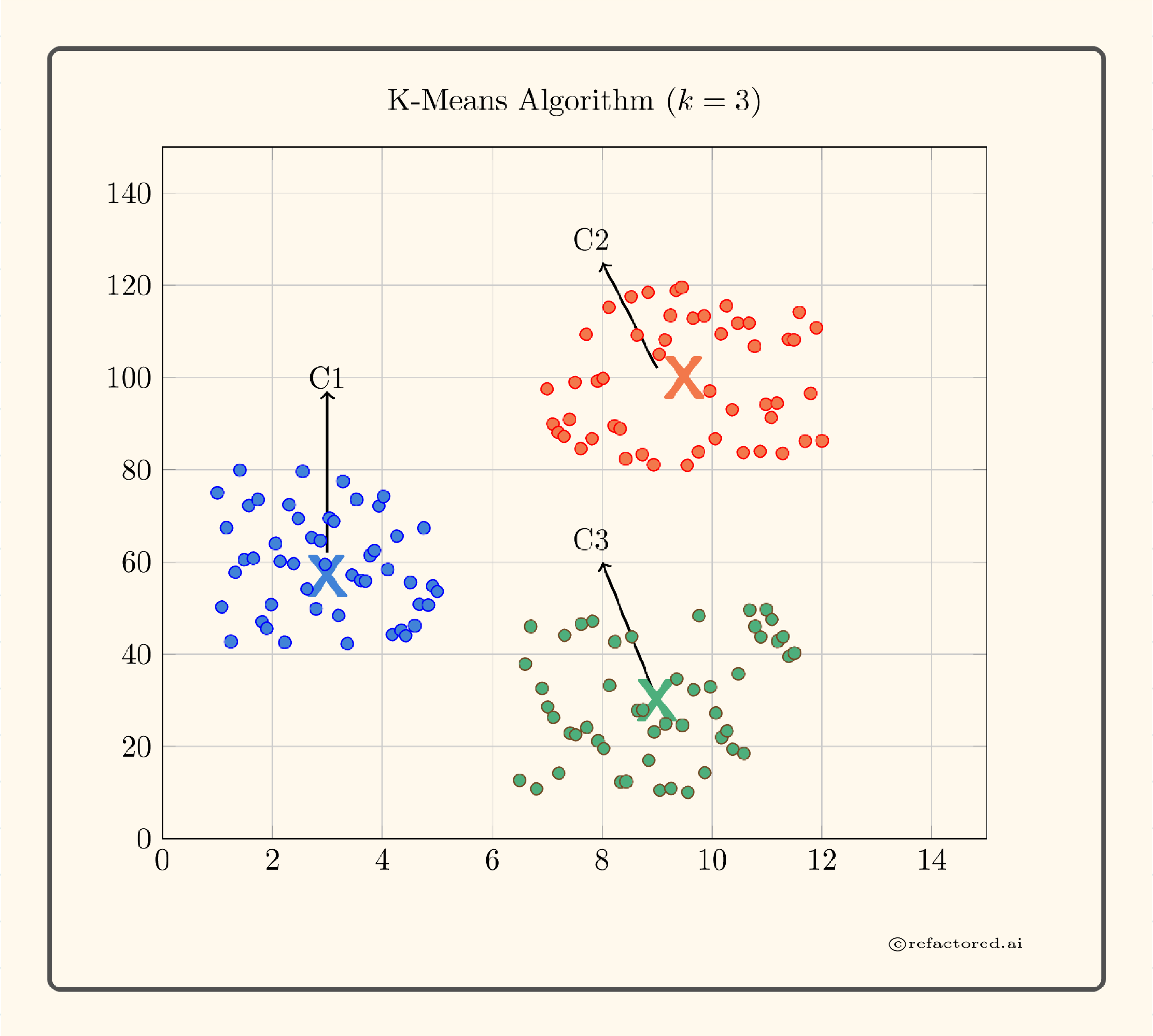
Portioning Based Clustering :: K-Means

- K-Means bekerja dengan memilih beberapa titik **data awal** (k) secara acak, lalu memindah-mindahkannya hingga pengelompokan yang paling ideal ditemukan
- K-Means adalah algoritma *clustering* yang populer untuk solusi berbagai bidang seperti: segmentasi gambar, segmentasi pelanggan, riset pasar, dan pengelompokan dokumen



Flowchart K-Means Clustering





Step Algoritma K-Means

- Step 1: Tentukan jumlah k Klaster
- Step 2: Secara acak pilih data point k sebagai titik pusat klaster
- Step 3: Gunakan persamaan Euclidean untuk menghitung jarak antara setiap titik data point dengan titik pusat klaster
  Distance = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- Step 4: Tentukan / set data point ke dalam klaster yang terdekat dengan data point
- Step 5: Hitung ulang titik pusat klaster yang baru terbentuk. Pusat cluster dihitung dengan mengambil rata-rata dari semua titik data yang terdapat dalam cluster tersebut

Step Algoritma K-Means

- Step 6: Terus ulangi prosedur dari step 3 s.d step 5 hingga salah satu kriteria penghentian berikut terpenuhi:
  1. Jika titik data berada dalam cluster yang sama
  2. Mencapai iterasi maksimum
  3. Klaster baru terbentuk tidak mengalami perubahan titik pusat

Contoh: Algoritma K-Means

- Terdapat titik-titik pada klaster berikut ini:
  P1(1,3) , P2(2,2) , P3(5,8) , P4(8,5) , P5(3,9) , P6(10,7) , P7(3,3) , P8(9,4) , P9(3,7).

- Step 1: Tentukan nilai K = 3

- Step 2: Tentukan inisial pusat klaster (***centroid***)
  Cluster-1 = C1 = P7(3,3)
  Cluster-2 = C2 = P9(3,7)
  Cluster-3 = C3 = P8(9,4)

Contoh: Algoritma K-Means

- Step 3: Hitung jarak antara data point dengan titik pusat klaster

Iterasi 1:

Untuk P1:

C1P1 =>(3,3)(1,3) => $sqrt[(1–3)^2+(3–3)^2]$ => sqrt[4] =>2

C2P1 =>(3,7)(1,3)=> $sqrt[(1–3)^2+(3–7)^2]$ => sqrt[20] =>4.5

C3P1 =>(9,4)(1,3) => $sqrt[(1–9)^2+(3–4)^2]$=> sqrt[65] =>8.1

Untuk P2:

C1P2 =>(3,3)(2,2) => $sqrt[(2–3)^2+(2–3)^2]$ => sqrt[2] =>1.4

C2P2 =>(3,7)(2,2)=> $sqrt[(2–3)^2+(2–7)^2]$ => sqrt[26] =>5.1

C3P2 =>(9,4)(2,2) => $sqrt[(2–9)^2+(2–4)^2]$=> sqrt[53] =>7.3

Lakukan Iterasi untuk semua data point

Contoh: Algoritma K-Means

- Step 4 & 5: Tentukan klaster dan *centroid* baru

Iterasi 1:

Data Points	Centroid (3,3)	Centroid (3,7)	Centroid (9,4)	Cluster
P1(1,3)	2	4.5	8.1	C1
P2(2,2)	1.4	5.1	7.3	C1
P3(5,8)	5.3	2.2	5.7	C2
P4(8,5)	5.4	5.4	5.1	C3
P5(3,9)	6	2	7.9	C2
P6(10,7)	8.1	7	3.2	C3
P7(3,3)	0	4	6.1	C1
P8(9,4)	6.1	6.7	0	C3
P9(3,7)	4	0	6.7	C2

Cluster 1 => P1(1,3) , P2(2,2) , P7(3,3)

Cluster 2 => P3(5,8) , P5(3,9) , P9(3,7)

Cluster 3 => P4(8,5) , P6(10,7) , P8(9,4)

Hitung mean (rata2) semua titik dalam klaster

New center of Cluster 1 => (1+2+3)/3 , (3+2+3)/3 => 2,2.7

New center of Cluster 2 => (5+3+3)/3 , (8+9+7)/3 => 3.7,8

New center of Cluster 3 => (8+10+9)/3 , (5+7+4)/3 => 9,5.3

Contoh: Algoritma K-Means

- Step 4 & 5: Tentukan klaster dan *centroid* baru

Iterasi 2:

Data Points	Centroid (2,2.7)	Centroid (3.7,8)	Centroid (9,5.3)	Cluster
P1(1,3)	1.0	4.5	8.3	C1
P2(2,2)	0.7	6.2	7.7	C1
P3(5,8)	6.1	1.3	4.8	C2
P4(8,5)	6.4	5.2	1.0	C3
P5(3,9)	6.4	1.2	7.0	C2
P6(10,7)	9.1	6.4	1.9	C3
P7(3,3)	1.0	5.0	6.4	C1
P8(9,4)	7.1	6.6	1.3	C3
P9(3,7)	4.4	1.2	6.2	C2

Cluster 1 => P1(1,3) , P2(2,2) , P7(3,3)

Cluster 2 => P3(5,8) , P5(3,9) , P9(3,7)

Cluster 3 => P4(8,5) , P6(10,7) , P8(9,4)

Hitung mean (rata2) semua titik dalam klaster

Center of Cluster 1 => (1+2+3)/3 , (3+2+3)/3 => 2,2.7

Center of Cluster 2 => (5+3+3)/3 , (8+9+7)/3 => 3.7,8

Center of Cluster 3 => (8+10+9)/3 , (5+7+4)/3 => 9,5.3

Contoh: Algoritma K-Means

- Step 3 – 5 : Hitung ulang jarak antara centroid baru K(C1,C2,C3) dengan data point

Iterasi 2:

C1(2,2.7) , C2(3.7,8) , C3(9,5.3)

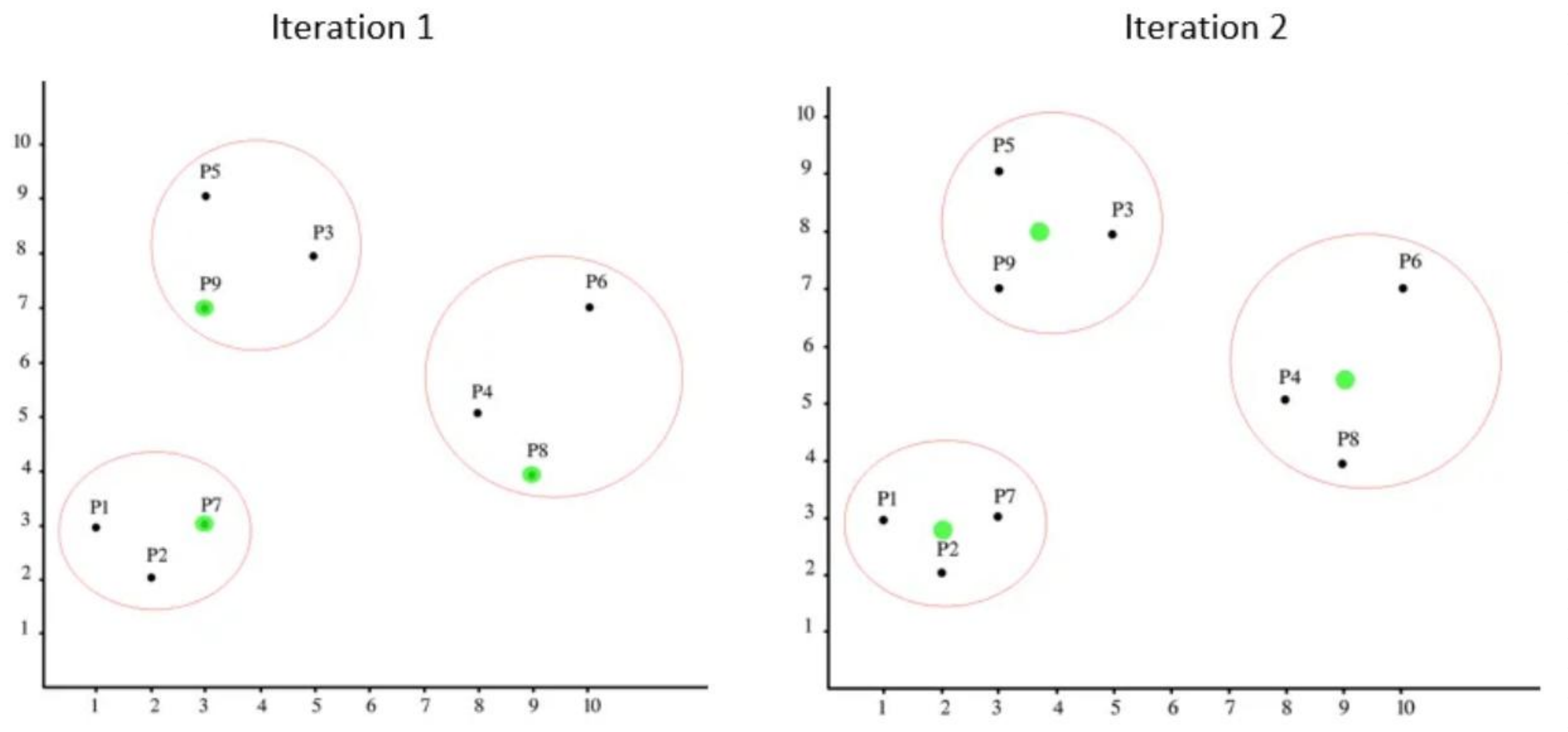
C1P1 =>(2,2.7)(1,3) => $\text{sqrt}[(1-2)^2+(3-2.7)^2]$ => sqrt[1.1] =>1.0

C2P1 =>(3.7,8)(1,3)=> $\text{sqrt}[(1-3.7)^2+(3-8)^2]$ => sqrt[32.29] =>4.5

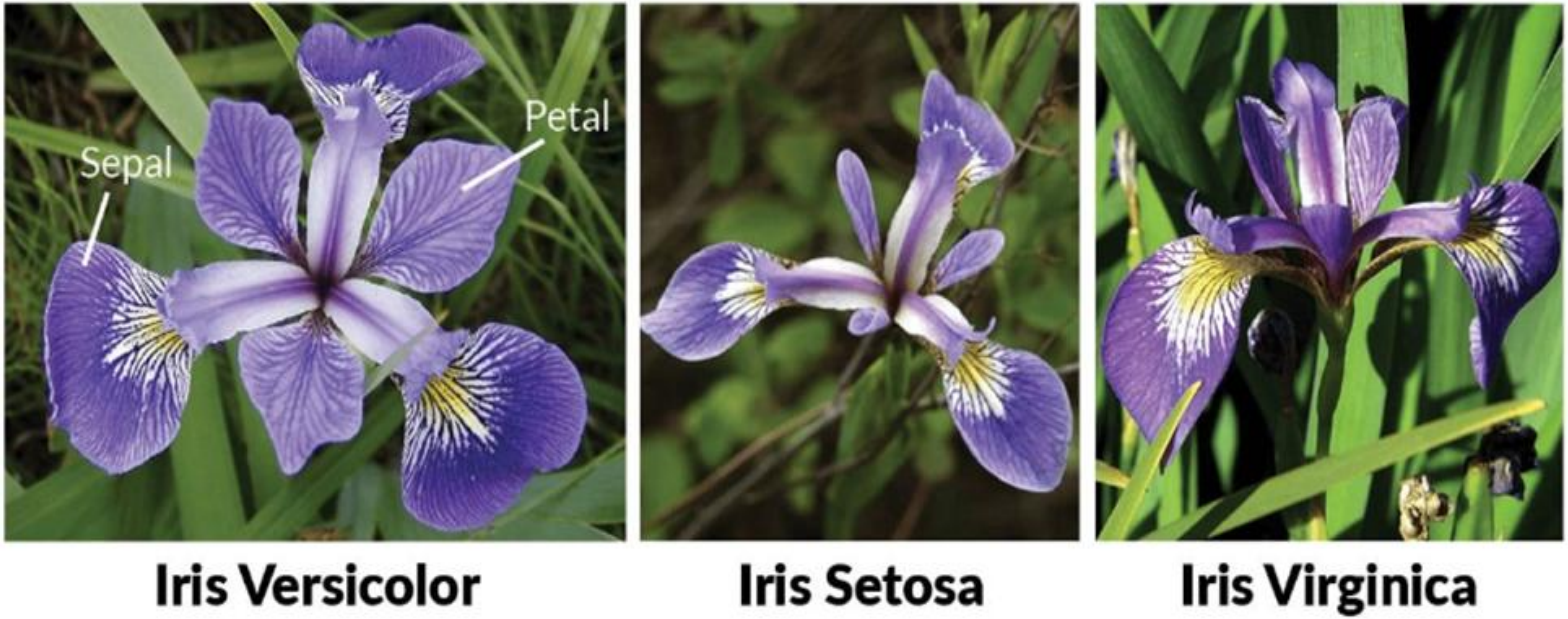
C3P1 =>(9,5.3)(1,3) => $\text{sqrt}[(1-9)^2+(3-5.3)^2]$=> sqrt[69.29] =>8.3

Lakukan Iterasi untuk semua data point

Hasil Iterasi 1 dan 2



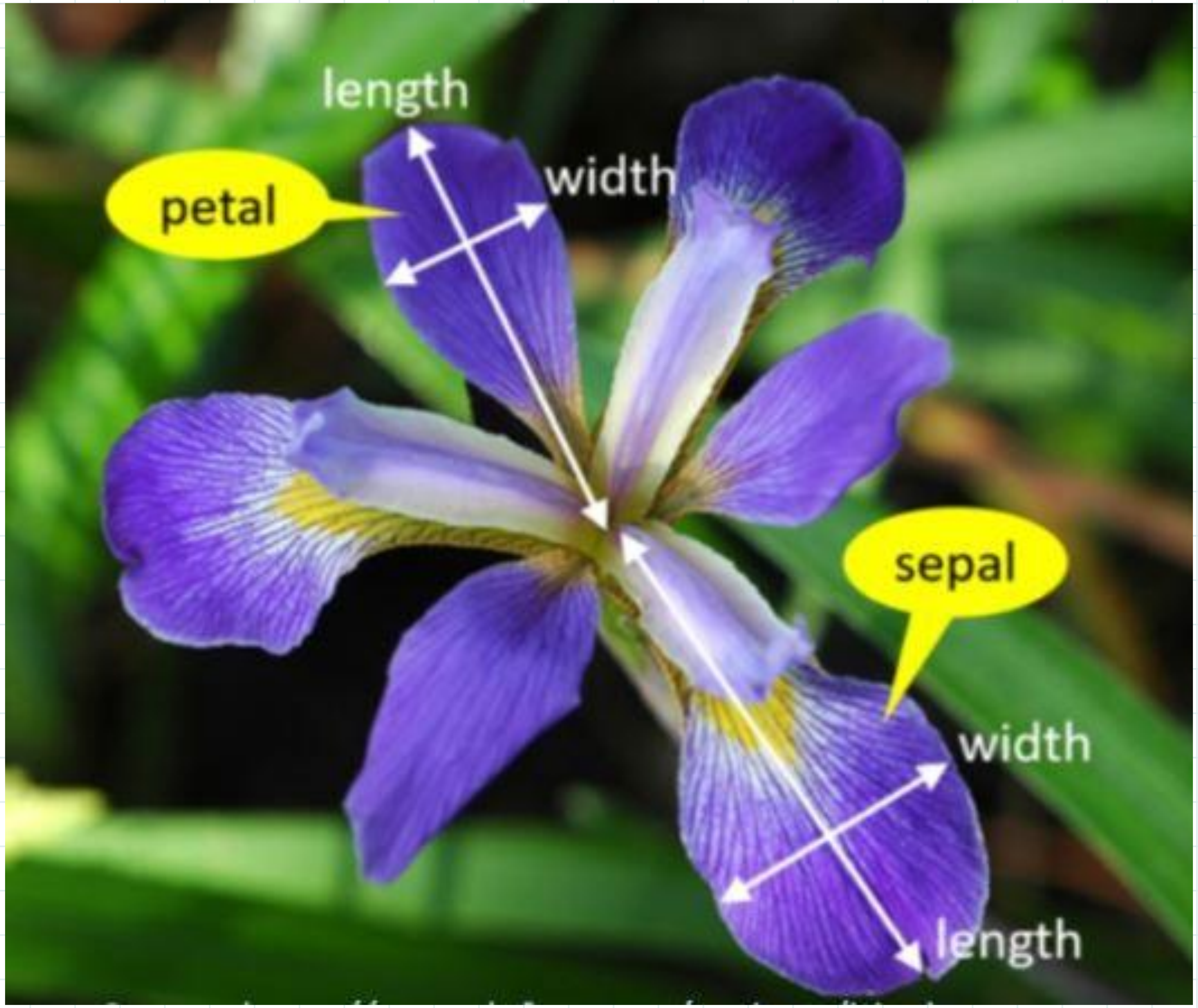
Case Study :: Dataset Iris



Dataset Iris :: Clustering

Attribute Dataset:
(1) Sepal Length, (2) Sepal Width
(3) Petal Length, (4) Petal Width
dalam satuan CM

Class Iris:
1. Setosa
2. Versicolour
3. Virginica



Sklearn.cluster :: KMeans

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
iris = pd.read_csv("dataset/iris.csv")
iris.info()
iris[0:10]
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

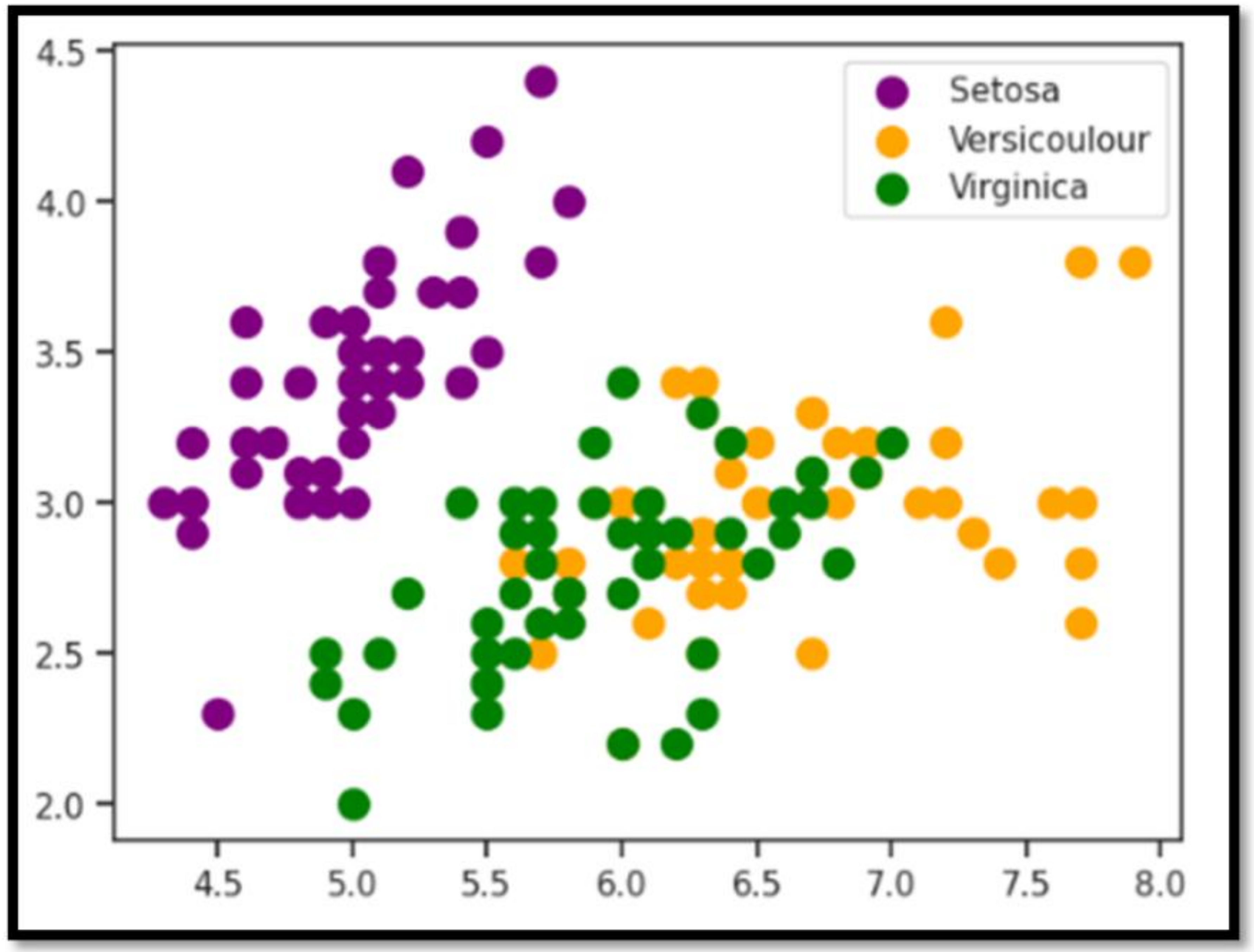
	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
5	5.4	3.9	1.7	0.4	Setosa
6	4.6	3.4	1.4	0.3	Setosa
7	5.0	3.4	1.5	0.2	Setosa
8	4.4	2.9	1.4	0.2	Setosa
9	4.9	3.1	1.5	0.1	Setosa

```
#Frequency distribution of species"
iris_variety = pd.crosstab(index=iris["variety"],  # Make a crosstab
                           columns="count")      # Name the count column
iris_variety
```

col_0	count
variety	
Setosa	50
Versicolor	50
Virginica	50

Sklearn.cluster :: KMeans

```python
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
encoder = OneHotEncoder()

categorical_data = iris[['variety']]
encoded_categorical = encoder.fit_transform(categorical_data).toarray()

encoded_df = pd.DataFrame(encoded_categorical,columns=encoder.get_feature_names_out(['variety']))

final_df = pd.concat([df_numeric,encoded_df],axis=1)

imputer = SimpleImputer(strategy='mean')
final_df_imputed = pd.DataFrame(imputer.fit_transform(final_df),columns=final_df.columns)

kmeans = KMeans(n_clusters=3,init='k-means++',max_iter=300,
               n_init=10,random_state=0)
y_kmeans = kmeans.fit_predict(final_df_imputed)

plt.scatter(final_df_imputed.iloc[y_kmeans==0,0],final_df_imputed.iloc[y_kmeans==0,1],s=100,c='purple',label='Setosa')
plt.scatter(final_df_imputed.iloc[y_kmeans==1,0],final_df_imputed.iloc[y_kmeans==1,1],s=100,c='orange',label='Versicolor')
plt.scatter(final_df_imputed.iloc[y_kmeans==2,0],final_df_imputed.iloc[y_kmeans==2,1],s=100,c='green',label='Virginica')
plt.legend()
plt.show()
```



https://www.linkedin.com/pulse/k-means-clustering-iris-dataset-hani-abudaba

Terima Kasih

http://youtube.com/@rojulman